Recommendation Algorithms: how they get what you want

CS 6220 – Big Data System & Analytics

Ruiyang Qin

Why Recommendation Algorithm is valuable to me and the readers

I want to start with a short story: James Lankford, a US Senator from Oklahoma, whose videos were popped up on my YouTube recommendation list. Maybe out of curiosity, I opened that video and realized that I will waste two minutes on boring political propaganda. However, the following comments were far more interesting than the video itself. The viewer said that they had never known this guy and they didn't understand why they suddenly received such a recommendation. There was one viewer commented, "**I guess I go wherever the algorithm wants me to go**."

As a computer science student, I might be able to answer their questions. During the National Election period, people watched too many politics-related videos and the algorithm might think that these people wanted to see more videos about politics. This might be the simplest explanation for the recommendation algorithm.

With the growth of the data volume, people are more and more dependent on the recommendation algorithms because it's nearly impossible to find what we want from billions of sources by manually searching. However, the increasing dependency means that we must land a part of our rights to the system and trust the system will unbiasedly recommend the appropriate materials to us.

If the recommendation system attempts to mislead us by recommending biased information, for example, then we might start to believe what it wants us to believe. Another example, TikTok's recommendation system. Because keep the users watching this app as long as possible will generate the best revenue, the recommendation system will keep feeding the users the videos they like. Different from YouTube, TikTok's system updates and analyzes its users' feedback more quickly and always trying to push only those videos that its users would like to see. The users might unintentionally spend three or four hours on that App per day.

After understanding how the recommendation system works, the users might have a different thought regarding these apps. They may try to avoid being over-controlled by the recommendation system or make the recommendation system knows them better. For example, since I know the recommendation system pictures my profile by analyzing my "following" channels, I could make the system recommend more the videos I like by only follow these channels that I am truly interested in.

In this paper, I am also going to explain the general framework of the recommendation system in the first section. Then I will use three popular companies as examples to show how the recommendation system work in the real world. In the last section, I will analyze and compare the performance of the three recommendation systems.

General Structure

In general, the recommendation system is a coordinator, working for the users, the environment, and the items. The users usually are people and sometimes are business groups. The environments include geographical features, time, and locations. This part usually will be wrapped up with the items. For example, YouTube will first recommend the latest hot videos instead of the videos posted 10 years ago. For items,

they could be videos, news, books, articles. Each of them has its features and can be labeled, groups based on their similarities. For example, the political news about the US-UK relationship will be grouped with other news about the Spanish election.

There are four features shared by the most recommendation system. The first feature is a relevant connection. For example, whether user A and user B are the same kinds of user, whether item 1 and item 2 could be classified as the same kind of item. This could be measured using the factorization machines model. An example from Rendle et al. [10], is showing in Fig. 2. The second feature has already been discussed. The third feature is the timing. For example, during the election of 2020, YouTube would recommend at least some news about presidential candidates. The fourth feature is the collaborative feature, which will be explained in the upcoming paragraph.

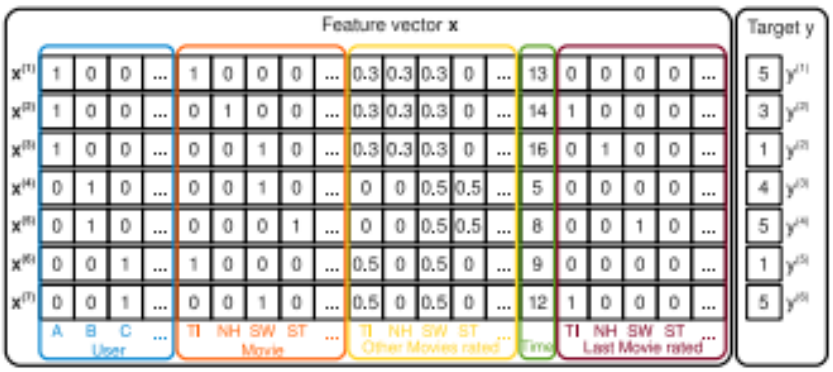
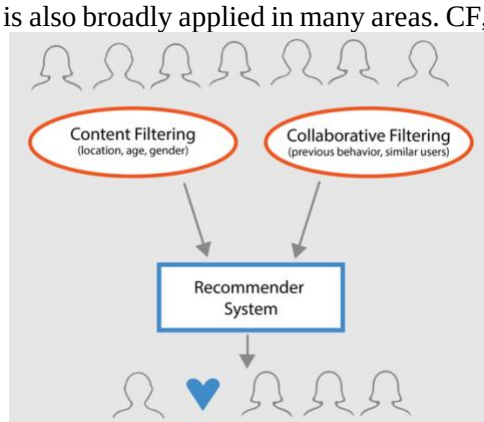


Figure 2

There are two main recommendation systems: **Content-based recommendation system** and **collaborative filtering recommendation system**. According to Pazzani et al. [8], the content-based recommendation system filters its items and pushes them to users based on the descriptions of the items and the profiles of the user interests. Many companies use this kind of recommendation system. By building up a personalized profile for each user, the system will be more and more capable of predicting users' preferences. The other different recommendation system, according to Schafer et al. [9], is also broadly applied in many areas. CF, stands for collaborative filtering, brings information, opinion, and feedback from connected networks together. CF is capable of deal with substantial quantities of data.



In this paragraph, I am going to discuss more details about the content-based recommendation system. The content-based recommendation system will filter the items that have similar features as those items the user preferred. The user's past choice, historical actions will be considered. In this process, the first thing is to calculate the similarity. A viable alternative is clustering. Assume each item has ten features. The matrix space is 10-dimensional. By clustering the features, the algorithm could figure out the items whose features are most overlapped. However, this method is time-consuming. Nowadays, most items could be generally labeled and classified. For example, a book could be classified into "education – history - American". In this case, the system could randomly pop up candidates from that classifying chunk. Both approaches share the basic idea. The second one is very similar to what Amazon does. To implement the content-based

recommendation system, the user profile is not necessary because the system could recommend only based on the user's current action. With the growth of the user profile, the recommendation could be more and more accurate.

In this paragraph, I am going to discuss more details about the collaborative filtering recommendation system. The recommendation system uses a concept called "confidence", which reflects how much possibility for one thing to happen in the future. At the beginning of this system, it collects feedback from different users. Those feedbacks can be seen as the user's interest. Then, based on their interests, the system could classify the users who share similar "interests". Based on the feedback from some users, the system could predict that the user from the same group might have similar feedbacks. Then it could generate a confident map similar to Fig. 1. The larger the number, the higher the user will prefer that item, where the y-axis represents the users and the x-axis represents the items.

.88	-1.08	0.9	1.09	-0.8
-0.9	1.0	-1.0	-1.0	0.9
0.38	0.6	1.2	-0.7	-1.18
-0.11	-0.9	-0.9	1.0	0.91

Figure 1.

<u>Algorithms Performance</u>

In this section, I am going to explain some implementations using the idea of collaborative filtering, content-based, and hybrid.

For Collaborative Filtering [11], the implementations are: ALS (Alternating Least Squares), A2SVD (Attentive Asynchronous Singular Value Decomposition), BPR (Bayesian Personalized Ranking), Caser (Convolutional Sequence Embedding Recommendation), FAST (FastAI Embedding Dot Bias), LightGCN, GRU4Rec, and Multinomial VAE.

For Content-based [11], the alternative implementations are: Deep Knowledge-Aware Network, Gradient Boosting Tree*, Neural Recommendation with Long- and Short-term User Representations, Neural Recommendation with Personalized Attention, Term Frequency - Inverse Document Frequency, Vowpal Wabbit, Wide and Deep, Factorization Machine & Field-Aware FM.

Let me take ALS algorithm to explain the details. In a short description, this algorithm is using matrix factorization to receive implicit feedbacks. It also makes optimization by Spark MLLib, in scalability and distributed computation. The core formula of matrix factorization is showing in the formula. The purpose of using the matrix factorization is to find the latent features which may demonstrate users' internal attributes and converts them into low dimensions. Here I am going to show a sample code from GitHub [11], where ALS is based on MovieLens recommender.

$$\min \sum c_{u,i}(p_{u,i} - q_i^T p_u)^2 + \lambda(||q_i||^2 + ||p_u||^2)$$

```python
import pyspark
from pyspark.sql import SparkSession
from pyspark.ml.recommendation import ALS
import pyspark.sql.functions as F
from pyspark.sql.functions import col
from pyspark.ml.tuning import CrossValidator
from pyspark.sql.types import StructType, StructField
from pyspark.sql.types import FloatType, IntegerType, LongType
```

```python
from reco_utils.dataset import movielens
from reco_utils.common.spark_utils import start_or_get_spark
from reco_utils.evaluation.spark_evaluation import SparkRankingEvaluation, SparkRatingEvaluation
from reco_utils.tuning.parameter_sweep import generate_param_grid
from reco_utils.dataset.spark_splitters import spark_random_split

print("System version: {}".format(sys.version))
print("Pandas version: {}".format(pd.__version__))
print("PySpark version: {}".format(pyspark.__version__))
```

```
System version: 3.5.5 |Anaconda custom (64-bit)| (default, May 13 2018, 21:12:35)
[GCC 7.2.0]
Pandas version: 0.23.0
PySpark version: 2.3.1
```

Data column names

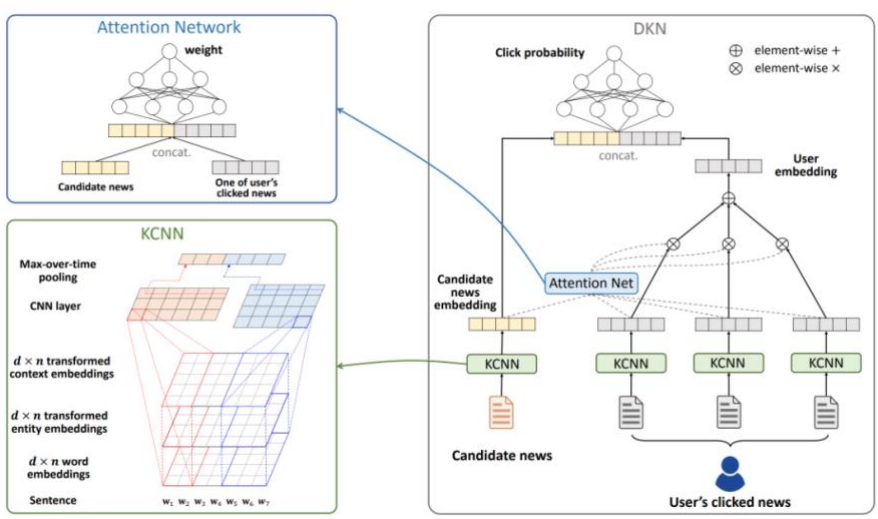
```python
COL_USER = "UserId"
COL_ITEM = "MovieId"
COL_RATING = "Rating"
COL_PREDICTION = "prediction"
COL_TIMESTAMP = "Timestamp"

schema = StructType(
    (
        StructField(COL_USER, IntegerType()),
        StructField(COL_ITEM, IntegerType()),
        StructField(COL_RATING, FloatType()),
        StructField(COL_TIMESTAMP, LongType()),
    )
)
```

Model hyper parameters - these parameters are selected with reference to the benchmarking results here.

```python
RANK = 10
MAX_ITER = 15
REG_PARAM = 0.05
```

The other implementation is called Deep Knowledge-Aware Network, or "DKN". This is a content-based filtering algorithm. The DKN is consisted of a knowledge graph and article embeddings. It uses them to conduct recommendations of news and articles. DNK is a deep learning model, used for CTR prediction instead of the traditional ID-based collaborative filtering. This algorithm uses the knowledge of entities and common sense from semantic-level and knowledge-level representation. It runs a dynamic calculation, just like most other recommendation systems do. To train this algorithm, people used a dataset called MIND, which is a large-scale English news dataset. Here I am going to show its architecture. DKN takes the sample news and the users' clicked news as input. It used the designed KCNN to process and generate the embedding vectors. This vector is used for the users' clicked history.



In the end, I am going to illustrate the performance of different algorithms, the calculated results shown in a table from [11]. The results is on Movielens 100k, and every algorithm run 15 epochs.

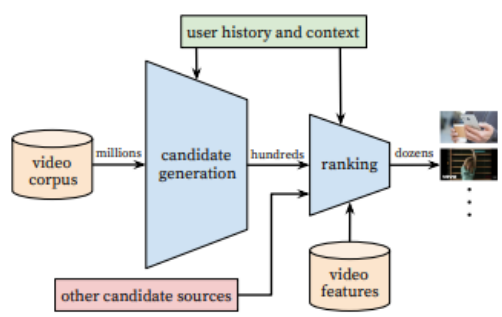
Algo	MAP	nDCG@k	Precision@k	Recall@k	RMSE	MAE	R^2	Explained Variance
ALS	0.004732	0.044239	0.048462	0.017796	0.965038	0.753001	0.255647	0.251648
SVD	0.012873	0.095930	0.091198	0.032783	0.938681	0.742690	0.291967	0.291971
SAR	0.110591	0.382461	0.330753	0.176385	1.253805	1.048484	-0.569363	0.030474
NCF	0.107720	0.396118	0.347296	0.180775	N/A	N/A	N/A	N/A
BPR	0.105365	0.389948	0.349841	0.181807	N/A	N/A	N/A	N/A
FastAI	0.025503	0.147866	0.130329	0.053824	0.943084	0.744337	0.285308	0.287671

Exemplar Recommendation System Applications

YouTube

Nearly fifteen years ago, around 2005 to 2012, YouTube valued the videos that gained more clicks. Because this is the beginning time of YouTube, it didn't apply complex algorithms. The user volume was also not as huge as its current user volume. The assumption is that the more clicks a video gained, the more popular it was.

After 2012, YouTube started considering the watch engagement, which was how much time people spent on each video and whether they made any comments. YouTube believed that watch time would be a good indicator of what people wanted to say. On its official website, YouTube explained why they focus on watch time. According to the official [1], "YouTube encourages people to spend more time watching, interacting, and sharing with the community," "If viewers are watching more YouTube, it signals to us that they're happier with the content they've found. It means that creators are attracting more engaged audiences."



How about the current recommendation system? Since 2016, YouTube has applied deep neural networks and machine learning in its recommendation system. There were many reasons to do that. According to the paper released by YouTube [2], there were three challenges to be dealt with during recommending videos: Scale, Freshness, and Noise. The first challenge was easy to understand. In 2016, YouTube video stock and its user body grew to an incredible level. A successful algorithm should not only be able to make a recommendation in time efficiently but find the appropriate videos. According to this paper, "Highly specialized distributed learning algorithms and efficient serving systems are essential for handling YouTube's massive user base and corpus." The second challenge was about how to balance the dynamic uploaded videos and users' latest response. As for the third challenge, it might be good to explicit that with an example of myself. As a YouTube user, I sometimes searched for

some algorithm teaching videos on YouTube. I also historically watched some videos whose types were not what I like today. Comparing to the videos I like, such as movie reviewing videos, the former information was just noise. I didn't want to receive algorithm teaching videos all the time, just because I searched for some of them.

There is an interesting paper [2] that talked about the recommendation system. According to the authors, "many of the interesting videos on YouTube have a short life cycle going from upload to viral in the order of days requiring constant freshness of recommendation." The recommended videos were generated based on users' personal preferences, such as liked, disliked, collected. Then, the recommended videos would be ranked to create some variety and diversity.

Whether the user chooses to view YouTube on the PC or mobile devices, they can only a limited number of videos, probably no more than twenty each time. For them, they expect to find some interesting videos. For example, if I use YouTube, at least I would expect to find two or three videos I like, for each time YouTube makes a recommendation. If the recommended list is always boring, the user might just lose interest to keep using this app. That's why the recommendation is not an easy job. The algorithm should filter out several videos from millions in a very short time, and it needs to do that for millions of users. That's why the new features of the system are vital.

A deeper look at this system, its structure is comprised of two neural networks, which are candidate generation and ranking. These two neural networks are perfectly matching the description of the last paragraph of the introduction part. For candidate generation, it filters a relatively small number of videos from the entire video body, based on the user history and context. However, the filtered video body is still too large to make an efficient recommendation. The filtered video chunk should be shrunken again into dozens of videos. That's where the feature of ranking comes to work. According to the authors, "Presenting a few 'best' recommendations in a list requires a fine-level representation to distinguish relative important among candidates with high recall." So, the ranking system will rank the candidates with their scores.

BiliBili

BiliBili, a Chinese video platform, is quite similar to YouTube in many aspects. For example, BiliBili is the most popular video platform in the Chinese language world, with the largest active users. Different from YouTube, BiliBili has a higher ratio of young/teenage users. Based on BiliBili's second-quarter 2020 financial results [4] released on August 27th, 2020, it has average monthly active users of 171.6 million and average daily active users of 50.5 million. According to one uncertified analysis [5], 78% of users are aged between 18 to 35, they have an average of 81 minutes of usage per day. At least partially it supports the previous point of view.

BiliBili, same as YouTube, consider user engagement and interaction as a key feature of the product. Dan Mu, a Chinese phrase that means flowing words on the screen, is a famous feature, labeling BiliBili as a peculiar product. Dan Mu also forms a culture that belongs only to BiliBili. However, why would I spend time introducing it? Because it is an important factor which BiliBili uses to measure user interaction. The more Dan Mu users launched in a video, the more popular this video will be recognized.

In YouTube, the algorithm measures user interaction and engagement only based on the number of comments, like or dislike, and watching time. Unlike YouTube, BiliBili cares less about the amount of watching and user interaction.

For BiliBili, it highly encourages the users to share some videos. If ordering the factors in weights, "sharing" is larger than "commenting", "commenting" is larger than "collect", and "collect" is larger than "watching time". BiliBili has never posted its algorithm structure. However, on April 22, 2019, one of its developers inadvertently leaked its source code [6]. Based on the analysis of this source code, we found the hotness formula:

$$video = coin * 0.4 + collection * 0.3 + barrage * 0.4 + play * 0.25 + like * 0.4 + share * 0.6$$

I In this formula, the coin is an additional like. In BiliBili's rule, each user will get a free coin each day. They can keep the coin and accumulate as many as they want. They also may give the coin to the videos. Different from "like", which is unlimited for anyone, "coin" is more valuable because its generating speed is limited. Barrage is "Dan Mu", as I've already explained.

The factor "share" has a comparatively high weight in this formula. I am going to explain some possible reasons. Unlike YouTube, BiliBili is still a start-up company, even if its volume is larger than many other contemporary companies. YouTube generates revenue by inserting advertisements into videos. That's partially why YouTube wants users to watch as long as possible. However, BiliBili promised to all its users that it will have insert advertisements. That's why the factor "play" only weights 0.25 in that formula.

For YouTube, it aims to create personalities. To achieve this goal, it must filter the appropriate candidates for each user from the millions of videos. However, BiliBili only recommends the hottest videos. Since there are several competitors in BiliBili's business environment, BiliBili needs to create a special community culture to keep its current users and attract new users. In some sense, BiliBili's users are sharing some common personalities. That's partially why BiliBili won't create a personality for each user. If a video is hot, it means this video is hot for all BiliBili's users. In this way, BiliBili only recommends hot videos.

To comprehensively understand its recommendation system, there is two another feature (factor, I am going to call factor as a feature in the following writing.) should also be considered: "following" and "label". From this perspective, YouTube does make some recommendations based on "personality". It will primarily recommend the hottest video in users' following list and their labels.

TikTok

On the official website [7], TikTok explained its recommendation system. In general, TikTok is more personalization-oriented than YouTube. The entire system aims to explore what the users like and what the users want to watch, feeding the users only videos they might be interested in. The algorithm won't intentionally influence its users' minds and recommend some political-biased videos. Neutrality is important for the recommendation system.

In the recommendation system, it builds up the user profile once the user creates the account and generates interactions, such as posting comments or following another account. Whether clicking like or skipping a video are also important features to be considered by the recommendation system.

According to TikTok official announcement, there are several things to be included to form the user's personalization: User interactions, Video information, Device, and account settings. In the previous paragraph, the feature of User interactions has already been explained. As for Video information, it includes hashtags, captions, and sounds. This feature is similar to the video label in BiliBili. The third feature, since TikTok is a multinational video platform, is used to provide the users with an appropriate community with which they might be more familiar.

Now, I am going to explain in more detail TikTok's model. TikTok uses a framework called Apache Storm to process user data. For Apache Storm, it is a distributed stream processing computation framework. This framework is in a topology structure, it can reliably process the unbounded streams of data, doing a good job dealing with real-time processing. Another tool TikTok uses is called Scribe, which is a log server, arranged in a directed graphic way. It is used to collect log data. Then the collected log will be imported into Apache Kafka. In short, this is a platform used to process the stream. From here, the data will be imported into Strom, updated the user model, and pushed into the storing system. This entire process is to calculate and update the user interests. Illustrated in the diagram:

$$\begin{aligned} &Collect\ the\ original\ data\ using\ \text{"Scribe"} \rightarrow Import\ Kafka\ file\ arrangement \\ &\rightarrow import\ into\ \text{"Storm"} \\ &\rightarrow Processing\ the\ baseline\ from\ high-speed\ storage\ system \\ &\rightarrow update\ the\ user\ model\ based\ the\ \text{"}Batch\text{"} \end{aligned}$$

This model, or we might call it as prototype, is widely used by many other platforms in China, such as XiGuaShiPin, another video platform.

<u>The outlook of the technology in my own words</u>

Nowadays, the technique of distributed systems is more and more widely applied. Combining with deep learning, from a technical perspective, the recommendation system itself will be more and more capable of understanding the users' true interests.

Recall the recommendation system used by YouTube, which filters candidates and ranking them. The system was designed for that structure partially because such an architecture will save limited computational power. Another example, calculating the similarity of items and users, using KNN or clustering, is also limited because of computational power. If there is a chunk of computers that can figure out a billion-sized computation (say 40 features of clustering computation) within 0.1 seconds, then the recommendation system might be changed.

The **change** would be the candidate filtering accuracy. The current recommendation systems, even if they find some things that the users might like, cannot be confident that every item they filter out that matches the users' expectations. To achieve a perfect matching, the recommendation system needs to fully understand what the users truly interested in and what are they truly potentially interested in. This could be achieved by deep learning techniques such as advanced convolutional neural network. Meanwhile, the recommendation system should also have high computational power to find the appropriate materials from the database. For platforms like TikTok, achieving a perfect matching is comparatively hard because the users highly frequently fresh the app

Cambridge Analytica

and the system needs to **real-timed** updating the users' profile. In such as liken dynamic environment, advanced computation is nearly impossible based on the current techniques.

Consequentially, with a comprehensive understanding of the users, the recommendation system would be able to achieve some other goals more than "recommending items". Taking this advantage, the recommendation system could manipulate the users' thoughts by sightly modifying the recommending contents. The only different that this advanced system from the current one is that this advanced system could do a much better job than the current recommendation system, given the situation that organizations like Facebook and Cambridge Analytica had used the user data to manipulate politics and the public views.

Potential Improvement could be made on the current Recommendation Systems:

While the recommendation systems helping people find things they might be interested in, such techniques build up an amorphous wall between people and the resources (videos, news, articles, songs). People can neither evaluate the quality of the recommended things nor fully explore on their own. In China, a company called Baidu, performed indecent business actions. This company, who took the advantages of recommendation system, to provide the users with false information to gain larger profits. In this way, the first improvement might be ***more stringent governmental regulations***. For example, the recommendation results should be mainly based on the quality of items themselves instead of advertisements.

From the technical perspective, adding some noisy factors is should a good idea. This so-called noisy factor is just some weights or biases to keep the recommendation system from knowing the users too much. For example, if a recommendation system knows what I like based on my personal profile and user history, it might be able to accurately pick up the things I like. However, my tastes may change, and I may want to see something different. However, I don't know what I might be interested in.

For the recommendation system, it should be able to make a prediction, within reasonable range of the things I might like. For example, I saw many file reviews videos (the video that YouTuber explains a film and leaves some comments.) in YouTube. In the old recommendation system, YouTube may only know that I like file review videos. Now, it should make some deeper analysis, and find out that I only watch sci-fi film review. Then, the recommendation system should give me some videos about sci-fi novels. In this way, ***the second improvement*** is: Deep learning on user behaviors and more detailed source classifications.

Discussion from various perspectives:

People always talk about freedom, human rights, and restricted governmental powers. However, the true potential threats may not actually come from the government, or the power itself. Just like the antitrust law's basic principle: having the monopoly power is not a crime but exerting that power to enrich the power holders themselves is a crime. With the continuous societal progression, we all have to rely on the techniques such as recommendation system to maintain and develop our life.

For everything in this world, a good regulation conducts better, an inferior regulation conducts the devil. Following this logic, we can safely assume that the recommendation systems will be able to understand people more and provide better services if they can be regulated. On the other hand, such techniques are easily used to manipulate the public, to jeopardize the political balancing, and even to create hatred between people. For example, if the recommendation system aims to keep the users in one app as long as possible, and it find out that hatred videos can achieve this goal, then should the recommendation system be allowed to keep recommend the users by the hatred videos? In my opinion, it's important to set a boundary and maintain the authenticity of the recommended things.

What I described above, may or may not happen. It's just a possibility. Maybe, in a different possibility, the recommendation system will be combined with other techniques such as AI and VQA, to create a better version of interaction interface (Better than Siri). Following this line, there are several interesting hypotheses we can make. The current recommendation system is based on user behavior analysis to try to predict the things that the user might like. However, with stronger computational power, the user might just communicate with their "recommender" directly. The users can simply describe what they want to have. The recommendation system, as long as it can understand the advanced level of language, will have a better idea of what the users really want. To implement that, the artificial intelligence must be further developed so that human language can be better understood.

Reference:

[1] https://youtube-creators.googleblog.com/2012/08/youtube-now-why-we-focus-on-watch-time.html

[2] Covington, P., Adams, J., & Sargin, E. (2016, September). Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems* (pp. 191-198).

[3] Davidson, J., Liebald, B., Liu, J., Nandy, P., Van Vleet, T., Gargi, U., ... & Sampath, D. (2010, September). The YouTube video recommendation system. In *Proceedings of the fourth ACM conference on Recommender systems* (pp. 293-296).

[4] https://ir.bilibili.com/static-files/29a668fc-77e7-4e02-83b0-58e1dc3134a8

[5] https://www.octoplusmedia.com/china-media-biliblii-video-platform-target-marketing-to-chinese-young-generations/#:~:text=Advertising%20Characteristics%20of%20Bilibili&text=80%25%20of%20users%20in%20Bilibili,with%20the%20highest%20consumption%20ability.

[6] https://www.programmersought.com/article/68264525511/

[7] https://newsroom.tiktok.com/en-us/how-tiktok-recommends-videos-for-you

[8] Pazzani, M. J., & Billsus, D. (2007). Content-based recommendation systems. In *The adaptive web* (pp. 325-341). Springer, Berlin, Heidelberg.

[9] Schafer, J. B., Frankowski, D., Herlocker, J., & Sen, S. (2007). Collaborative filtering recommender systems. In *The adaptive web* (pp. 291-324). Springer, Berlin, Heidelberg.

[10] Rendle, S. (2010, December). Factorization machines. In *2010 IEEE International Conference on Data Mining* (pp. 995-1000). IEEE.

[11] https://github.com/microsoft/recommenders